Regis University CPS
CS361 Programming Assignment 4
due by midnight Sunday of week 4

Program #1

A department store would like you to write a program to compute the bills for an upcoming sale. You will also write a test plan for this program.

The store is offering discounts to customers, based on the customer type and amount of purchase. The following discounts will be offered:

Total amount of purchase	General-Public Customer	Store Rewards Club Member	Store Credit Card Holder
Under 50	3%		20% + additional 5% if Gold Card
50 to under 100	5%	7%	
100 to under 200	8%	12%	
200 or over	16%	20%	

NOTES:

1. **All** of the fixed values in the table should be stored as **constants**, using descriptive constant names. This means that you will need to create constants for the **purchase limit** dollar amounts in each bracket, as well as for the **rates** in each bracket.

2. The store should be able to modify any of the figures, simply by changing the constant values, and without having to modify any of the executable code.

3. Do not embed any constant values in the constant names!

Example:

```
const double CLUB_RATE_50_OR_OVER = 0.07;        // incorrect
```

The above constant is defined incorrectly, because the purchase minimum of 50 is embedded in the constant name. Instead, a separate constant should be defined to hold the minimum:

```
const int MIN_PURCH_RATE2 = 50.00;
const double CLUB_RATE2 = 0.07;                  // better!
```

Program Implementation

The program should first describe its purpose to the user. Then it should prompt for and read the customer type and purchase amount from the user.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Input Example 1

```
Department Store Sale Bill Calculator

Customer Type Choices:
    P – general public
    R – rewards club member
    C – credit card holder

Enter customer type: R

Enter total amount of purchase: 55.44
```

Note: The program does not have to error check any of the user input -- you may assume all valid answers will be entered. But the program must accept both upper and lowercase entries of any character input.

If the customer is a credit card holder, the program should also prompt for the credit card type.

Input Example 2

```
Department Store Sale Bill Calculator

Customer Type Choices:
    P – general public
    R – rewards club member
    C – credit card holder

Enter customer type: c

Enter total amount of purchase: 144

Credit card type (S for standard, G for gold): g
```

After reading all necessary input, the program will call a **user-defined function** to determine which **discount rate** to apply.

The user-defined function will:

- ✓ Be given a descriptive name that describes what the function does
- ✓ Take **customer type** and **purchase total** as input arguments
    - Possible customer types that can be passed in are:
        - P = general-public
        - R = rewards club
        - S = standard card holder
        - G = gold card holder
- ✓ Return a **discount rate** to use in the bill calculations in the ***main*** function.
- ✓ Use BOTH nested **if** statement(s) AND extended multi-way selection **if** statement(s) to determine the correct discount rate for the customer.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

NOTE: No **switch** statements shall be used in this function.

The **main** function will use the returned **discount rate** to compute the **customer's discount**.

After applying the discount, the customer will be charged sales tax on their purchase, at the sales tax rate of **7.5%.** Finally, the program should neatly display the customer type, discount rate, and bill details.

Output Example 1 (using Input Example 1 data)

```
Department Store Bill

  Customer Category:          Rewards Club
  Discount Rate:                        7%

  Total Purchase Amount:        $    55.44
  Less Discounted Amount:       $     3.88
                                ----------
  Subtotal after discount:      $    51.56
  Plus Sales Tax (7.5%)         $     3.87
                                ----------
  Total Bill:                   $    55.43
```

Notes on output:

1. The customer type should be displayed in words (not just a character) and line up on the right with the dollar figures below.
2. Rate used should be displayed as a whole number and line up on the right with the dollar figures below.
3. All dollars and cents amounts should be displayed to 2 decimal places and aligned on the decimal point.

Output Example 2 (using Input Example 2 data)

```
Department Store Bill

  Customer Category:      Gold Card Holder
  Discount Rate:                       25%

  Total Purchase Amount:        $   144.00
  Less Discounted Amount:       $    36.00
                                ----------
  Subtotal after discount:      $   108.00
  Plus Sales Tax (7.5%)         $     8.10
                                ----------
  Total Bill:                   $   116.10
```

Testing

As part of your submission for this week, you must write a **test plan** for this program.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Your test plan must include both ***rationale*** for testing all conditions within the program and a list of the specific test cases. Each test case must include the exact ***user input used*** for the test and what the exact ***output expected***.

A test plan for one of last week's programs is included in the online Content as a sample (see section 4.3.3).

The test plan should be contained in either a Word doc or an Excel spreadsheet.

NOTE: Although you are required to ***turn in*** a test plan for ***only this*** program, you will still be expected write your own test cases to test and validate the code of the other program, validating ***all*** possible logic paths.

Sample Plan Format

Rationale:

1. Test uppercase entry of customer type
2. Test purchase amount at lower boundary for second bracket

:

Input Values Tested			**Output Expected**							**Comments**
Customer Type	**Card Type**	**Purchase**	**Customer Type**	**Rate Used**	**Purch**	**Discount**	**Subtotal**	**Tax**	**Total**	
P	n/a	50.00	General Public	5%	50.00	2.50	47.50	3.56	51.06	
:		:	:	:						:

You may submit your test plan as an MS Word document or an MS Excel spreadsheet.

Program #2

A partial list of undergraduate and graduate programs, similar to what is offered within the School of Computer and Information Sciences at Regis University, is shown below:

Program Name (abbreviation)	Earns
Business Technology Management (BTM)	BS
Computer Information Systems (CIS)	BS
Computer Networking (CN)	BS
Computer Science (CS)	BS
Database Technologies (DT)	MS
E-Security (ES)	Certificate
Information Assurance (IA)	MS
Information Technology Management (ITM)	MS
Storage Area Networks (SAN)	Certificate
Software Engineering Practices (SEP)	MS
Systems Engineering Technologies (SET)	MS

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

You will write a program to identify the program, using 1 to 3 input letters from the program abbreviation.

The program should first describe its purpose.

Then prompt the user for the first letter in the program abbreviation (i.e. the first letter in the first word of the program name).

Use **one** large ***nested*** **switch** statement to output one of the following messages:

- If the letter is 'B' or 'b', output: BS in Business Technology Management (BTM)
- If the letter is 'D' or 'd', output: MS in Database Technologies (DT)
- If the letter is 'E' or 'e', output: Certificate in E-Security (ES)
- If the letter is 'C' or 'c', prompt for a second letter in the program abbreviation (i.e. the first letter of the second word in the program name).
    - If the second letter is 'I' or 'i', output: BS in Computer Information Systems (CIS)
    - If the second letter is 'N' or 'n', output: BS in Computer Networking (CN)
    - If the second letter is 'S' or 's', output: BS in Computer Science (CS)
- If the letter is 'I' or 'i', prompt for a second letter in the program abbreviation (i.e. the first letter of the second word in the program name).
    - If the second letter is 'A' or 'a', output: MS in Information Assurance (IA)
    - If the second letter is 'T' or 't', output:
      MS in Information Technology Management (ITM)
- If the letter is 'S' or 's', prompt for the next two letters in the program abbreviation to be entered together (note that you can still read them into two separate character variables). Be sure to verify the correctness of BOTH the second and third letters (**not** just the third letter!)
    - If the second letter is 'A' or 'a', check the third letter (the second letter of the first word in the program name).
        - If the third letter is 'N' or 'n', then output:
          Certificate in Storage Area Networks (SAN)
    - If the second letter is 'E' or 'e', check the third letter (the second letter of the first word in the program name).
        - If the third letter is 'P' or 'p', then output:
          MS in Software Engineering Practices (SEP)
        - If the third letter is 'T', or 't', then output:
          MS in Systems Engineering Technologies (SET)
- If **none** of the above situations apply, output: Unknown Program

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

The program should ***always*** produce an output message. Therefore, you need to insure that ALL possible input combinations produce output of ***either*** a correct Program Name, ***or*** the "Unknown Program" message.

Notes for Program 2:

1. The program should never require the user to enter more than three letters to determine the output.
2. You are **not** allowed to use *any* **if** statements in this program.
3. There will be no user-defined functions in this program – just a **main** function.

General Notes for BOTH programs:

1. Remember to start on paper, with pseudo code or a flow chart!
2. Program logic must be **indented correctly**.
3. Comments must be included for any constant/variable whose name is not completely descriptive.
4. Include top of program comments as specified in course Content section 1.8.
5. Provide a comment above each user-defined function that includes:
   - What the function does
   - The name and a description of the parameters that are input to the function
   - A name (if available) and a description of what is returned from the function

   Example:

```
// -------------------------------------------------------
// Description: Function will sum two numbers
// Input parameters: num1 – first operand
//                   num2 – second operand
// Returns:  sum - sum of num1 and num2
// -------------------------------------------------------
int SumTwo (int num1, int num2)
```

Submission

This programming assignment is due by midnight Sunday. Submit your program source code to the ***Prog Assn 4*** folder in the ***Dropbox*** area of the online course. **All** files should be attached to **one** submission for programming assignment 4.

Before submitting your program files, you MUST name them as follows:

```
Lastname-wk4-prog1.cpp
Lastname-wk4-prog2.cpp
```

And your test plan file name MUST be:

```
Lastname-wk4-p1testplan.xxx
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

For example:

Smith-wk4-prog1.cpp
Smith-wk4-prog2.cpp
Smith-wk4-p1testplan.xls

Grading

The rubric that will be used to grade your program is included on the next pages.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

CS 361 Control Structures
Programming Assignment 4 Rubric

Program 1: (53 pts)				
Rating **Rating Category**	**Exemplary**	**Proficient**	**Basic (needs work)**	**Not Demonstrated**
Data Storage Points	Constants defined for ALL fixed values with no values embedded in names. All variables defined locally, using appropriate data types. Followed coding standards naming conventions.	A few minor errors in data typing, naming, or assignment OR missing one or two necessary constants for fixed value.	One or more major errors, OR majority of names did not follow naming conventions, OR missing multiple constants for fixed values.	Constants not used, identifier names are not descriptive, & there is a multitude of errors OR **used one or more global variable.**
Program Input	Displays descriptive prompt for each input and reads required input correctly. Recognizes upper/lowercase char input. Includes logic so no unnecessary input is read.	Reads the required input correctly, but does not display an exacting prompt OR reads unnecessary input.	Reads only some of the required input incorrectly, OR does not read some required input at all.	Hardcoded values used instead of reading input, OR reading of input fails completely.
User-defined Function Definition and Call	Function defined and called as specified by the requirements, with proper parameter passing. Function called correctly and returned value is captured correctly.	One or two minor errors in function definition or function call.	Many errors in function definition or call, or value not returned and saved correctly.	Required function was not implemented. Task was simply performed from within the main function.
User-defined Function Logic: nested/extended ifs	Used BOTH extended if statements and nested if statements, all written correctly. Uses constants for ALL fixed values. Correct value is determined.	Minor error(s) in the function logic or constant usage.	Many errors in the function logic or constant usage, and/or used **only extended** or **only nested** if statements, **not both.**	Function logic fails completely.
main Function Logic: if statements and calculations	The program logic accomplishes all program requirements and uses constants for all fixed values. All math computations compute correct results. All if statements written correctly.	One or two minor errors in the program logic, including incorrect if statement logic or incorrect math computations or lack of constant usage.	Multiple errors in the program logic, including incorrect if statement logic or incorrect math computations or lack of constant usage.	Logic accomplishes few or none of the program requirements.
Program Output	Displays all of the required output and follows required output format, according to program requirements	Displays all the required output, but there are one or two minor errors in the output format.	Does not display some of the required output, OR displays all of the required output, but has multiple errors in the output format.	Displays completely incorrect output, or does not display any output at all.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.
Program 2: (42 pts)				
Data Storage Points	All variables defined locally, using appropriate data types. Used descriptive names, following coding standards naming conventions.	One or two minor errors in data typing, naming, or assignment.	One or more major errors, OR majority of names did not follow naming conventions.	None of the identifier names are descriptive, there is a multitude of errors and/or defined global variable(s).
Program Input	Displays a descriptive prompt for each input. Reads input as detailed in requirements. No unnecessary input read.	Reads input correctly, but does not display exacting prompts, OR does not read some of the input as detailed in requirements, OR reads one unnecessary input.	Some input read incorrectly, OR reads multiple unnecessary input items.	Hardcoded values used instead of reading input, OR reading of input fails completely.
Output	Correct values are displayed, formatted as specified by the requirements.	There is a minor error in the output content and/or format.	Multiple errors in the output content and/or format, or missing output in some cases.	Displays completely incorrect data output, or does not display any output.
C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.
Test Plan (5 pts)				
Test Plan	Test plan includes rationale and tests that validate ALL logic.	Missing parts of rationale or missing tests of some logic.	Missing most or all rationale and/or missing many tests of the logic.	No test plan submitted.

Documentation	Top of program comments include all required components. Top of function comments explain what function will do, and describe all parameters and return values. Inline comments are included as necessary.	Program description missing some details about the input, output, or processing OR top of function comments vague or missing some details about parameters and return values OR missing other minor necessary comment.	Documentation has many problems with missing, incomplete and/or incorrect items in program description, top of function comments, and/or inline comments.	Only a few (or no) comments in program.

NOTE: Deductions for late submissions will be applied (-2% per day late)
But programs will **not be accepted** if they are submitted **more than 5 days** late (grade will be 0).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law